

A Comparative Study of Naive Bayes, Logistic Regression, and SVM for Spam Email Classification on the Spambase Dataset

Kaustubha Venkata Eluri
Department of Computer Science
Northeastern University
CS 5002 Project

Abstract—This paper documents a classical machine learning pipeline for spam email detection built on the 1999 UCI Spambase dataset (4,601 rows, 57 engineered features). Four classifiers are compared: a hand-rolled Naive Bayes implementation using log-space arithmetic and Laplace smoothing, a scikit-learn Bernoulli Naive Bayes model, Logistic Regression, and a Support Vector Machine (SVM). On the Spambase dataset, Logistic Regression achieves the best overall performance with 0.9294 accuracy, 0.9209 precision, 0.8981 recall, an F1 score of 0.9093, and an AUC-ROC of 0.9702, closely followed by SVM (0.9251 accuracy, 0.9688 AUC-ROC). The hand-rolled Naive Bayes classifier, evaluated separately, attains high recall (0.9890) but comparatively low precision (0.6950), illustrating the classic precision/recall tradeoff across model families. A separate raw-text pipeline evaluated on 1,035 emails (286 spam, 749 ham) achieves 97% precision and 95% recall. We report the exact metrics documented in the project, describe the underlying methodology, and discuss the limitations of this dataset for modern spam filtering.

Index Terms—spam classification, Naive Bayes, logistic regression, support vector machine, Spambase, text classification

I. INTRODUCTION

Spam email filtering is a canonical problem in applied text classification and a standard pedagogical vehicle for teaching classical machine learning pipelines: feature extraction, probabilistic modeling, discriminative classification, and quantitative evaluation. This project, developed as coursework for CS 5002 at Northeastern University, implements and compares four classifiers for spam detection: a hand-rolled Naive Bayes classifier implemented from first principles, and three scikit-learn models (Bernoulli Naive Bayes, Logistic Regression, and SVM) evaluated on the UCI Spambase dataset.

The motivation for including a hand-rolled Naive Bayes implementation alongside library-based models is explicitly pedagogical: hiding the algorithm behind a single `clf.fit(X, y)` call obscures the mechanics of Bayesian inference, log-space probability arithmetic, and additive smoothing that make the algorithm work in practice. By re-deriving these steps manually and comparing the result against scikit-learn’s implementations and against discriminative alternatives (Logistic Regression, SVM), the project surfaces both the mathematical foundations of Naive Bayes and the practical tradeoffs between model families.

II. RELATED WORK

Naive Bayes classifiers have long been used as a baseline for spam filtering because word-occurrence statistics differ sharply between spam and legitimate (“ham”) email, and the conditional-independence assumption, while technically violated for natural language, tends to work well in practice for this bag-of-words setting. More broadly, spam detection is a well-studied instance of binary text classification, and it is commonly used to introduce the standard classical machine learning pipeline: text-to-feature vectorization, probabilistic or discriminative model training, and evaluation via accuracy, precision, recall, F1, and ROC-based metrics. The Spambase dataset, a widely used UCI benchmark of 4,601 emails represented by 57 numeric features (word-frequency, character-frequency, and capital-run-length statistics), is a standard reference dataset for exactly this kind of classical spam-classification study.

III. METHODOLOGY

A. Hand-Rolled Naive Bayes

The manual Naive Bayes implementation (`data_import.py` in the project) applies Bayes’ theorem directly. For a document represented by its tokens, the posterior probability of class $c \in \{\text{spam}, \text{ham}\}$ given the observed words $w_1, \dots, w_n$ is

$$P(c \mid w_1, \dots, w_n) \propto P(c) \prod_{i=1}^n P(w_i \mid c), \quad (1)$$

where $P(c)$ is the class prior estimated from the training-set class distribution, and $P(w_i \mid c)$ is the conditional probability of word w_i occurring in class c , estimated from training-set word frequencies.

a) *Log-space arithmetic.*: Multiplying on the order of 10,000 token probabilities directly causes numerical underflow, since each individual probability is small and floating-point precision cannot represent the resulting product. The implementation therefore works in log-space:

$$\log P(c \mid w_1, \dots, w_n) \propto \log P(c) + \sum_{i=1}^n \log P(w_i \mid c), \quad (2)$$

replacing the product of probabilities with a sum of log-probabilities, which is numerically stable, and classification proceeds by comparing $\log P(\text{spam} \mid \cdot)$ against $\log P(\text{ham} \mid \cdot)$.

b) Laplace smoothing.: Because a word that never appears in the training data for a class would otherwise assign that class a conditional probability of exactly zero — forcing the entire posterior to zero regardless of other evidence — the implementation applies Laplace (add-one, $\alpha = 1$) smoothing to every word-count estimate, ensuring unseen words receive a small nonzero probability rather than collapsing the posterior.

B. Scikit-learn Model Comparison

In addition to the hand-rolled classifier, the project trains and evaluates three scikit-learn models on the Spambase feature set: Bernoulli Naive Bayes, Logistic Regression, and a Support Vector Machine (SVM). Feature extraction for the raw-email pipeline uses a bag-of-words representation built with `CountVectorizer()`, producing a document-term matrix of word frequencies; the Spambase dataset itself is already represented as 57 precomputed numeric features (word-frequency, character-frequency, and capital-letter-run statistics) rather than raw text.

IV. EXPERIMENTAL SETUP

Two datasets are used in this project, serving two distinct evaluation paths. The Spambase dataset (4,601 rows, 57 engineered features) is used for the multi-model comparison study across Manual Naive Bayes, Bernoulli Naive Bayes, Logistic Regression, and SVM. A separate raw-text corpus of 1,035 emails (286 spam, 749 ham) is used to evaluate the end-to-end Multinomial Naive Bayes pipeline operating directly on email text.

For the Spambase comparison, the dataset is divided into training (75%) and testing (25%) subsets, with a fixed train/test split (`test_size=0.25`, `random_state=42`) for reproducibility. Models are additionally evaluated with 5-fold cross-validation. Test suites (16 unit tests) cover vectorization correctness and the manual Naive Bayes log-probability arithmetic, but deliberately do not assert specific accuracy numbers, since sklearn version drift could make such assertions flaky.

V. RESULTS

A. Raw-Text Pipeline

Evaluated on the 1,035-email test corpus (286 spam, 749 ham), the Multinomial Naive Bayes model achieved 97% precision and 95% recall, with a confusion matrix of True Spam: 273, True Ham: 740, False Spam: 9, False Ham: 13.

B. Spambase Multi-Model Comparison

Table I reports the documented per-model metrics on the Spambase dataset (4,601 emails).

Logistic Regression obtains the highest accuracy (0.9294), AUC-ROC (0.9702), and cross-validation accuracy (0.9246 ± 0.0068) of the four models. SVM is a close second on nearly every metric. The Manual Naive Bayes classifier and Bernoulli

TABLE I
MULTI-MODEL COMPARISON ON THE SPAMBASE DATASET (4,601 EMAILS)

Model	Accuracy	Precision	Recall	F1
Manual NB	0.8243	0.6950	0.9890	0.8163
Bernoulli NB	0.8762	0.8716	0.8044	0.8367
Logistic Reg.	0.9294	0.9209	0.8981	0.9093
SVM	0.9251	0.9224	0.8843	0.9030

TABLE II
AUC-ROC AND CROSS-VALIDATION ACCURACY

Model	AUC-ROC	CV Accuracy
Manual NB	—	—
Bernoulli NB	0.9496	0.8868 ± 0.0081
Logistic Reg.	0.9702	0.9246 ± 0.0068
SVM	0.9688	0.9213 ± 0.0062

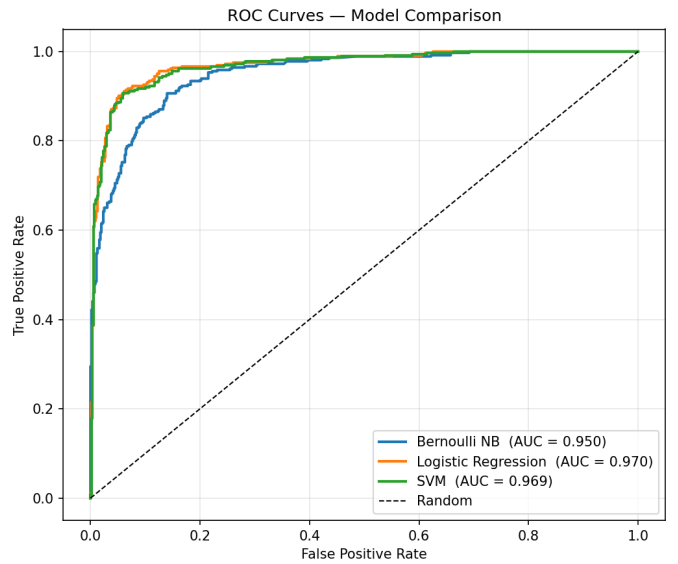


Fig. 1. ROC curves for the scikit-learn models evaluated on the Spambase dataset. Logistic Regression and SVM achieve the highest AUC-ROC (0.9702 and 0.9688, respectively), visibly dominating Bernoulli NB (0.9496) across the ROC space.

NB were not evaluated for AUC-ROC or cross-validation accuracy in the documented results.

Fig. 1 shows the ROC curves underlying the AUC-ROC values reported in Table II, illustrating that Logistic Regression and SVM maintain a favorable true-positive/false-positive tradeoff across nearly the full range of decision thresholds.

Fig. 2 visualizes the per-class error counts behind the precision and recall values in Table I, making explicit the precision/recall tradeoff discussed further in Section VII.

VI. DISCUSSION

The results in Table I illustrate a clear precision/recall tradeoff across model families. The hand-rolled Manual Naive Bayes classifier achieves the highest recall of all four models (0.9890) but the lowest precision (0.6950): it catches nearly



Fig. 2. Confusion matrices for the four classifiers on the Spambase test split. The Manual Naive Bayes matrix shows a comparatively high false-positive count consistent with its low precision (0.6950), while Logistic Regression and SVM show more balanced error distributions.

all spam but at the cost of frequently misclassifying legitimate email as spam. Bernoulli NB, Logistic Regression, and SVM shift this balance toward precision, with Logistic Regression and SVM achieving precision above 0.92 while sacrificing some recall relative to Manual NB. As documented in the project, this is a genuine multi-model comparison rather than a search for a single winning model: each model makes a different precision/recall tradeoff, and the appropriate choice depends on which class of error — a missed spam email (false negative) or a legitimate email misrouted to spam (false positive) — is more costly in a given deployment context.

VII. LIMITATIONS

As documented in the project’s limitations notes, several caveats apply. The Spambase dataset is from 1999; spam techniques have evolved substantially since then, and the model cannot see modern spam characteristics such as images, polymorphic content, or adversarial perturbations. The raw-text corpus used for the end-to-end pipeline is small (1,035 emails), not necessarily representative of production traffic distributions, and English-only, with no multilingual evaluation. The feature representation is bag-of-words only, with no n-grams, character-level features, embeddings, or header/metadata features (e.g., sender reputation, SPF/DKIM/DMARC), unlike modern neural or header-aware spam filters. No hyperparameter tuning or model calibration was performed; default scikit-learn hyperparameters were used throughout, and reported probabilities are not calibrated. Evaluation relies on a single train/test split plus 5-fold cross-validation, without stratification enforcement beyond the defaults of `train_test_split`, and no latency or memory profiling was conducted. The system is not deployed and has no retraining schedule, drift monitoring, incremental learning, or false-positive cost weighting; it is explicitly framed as a classical-ML coursework project rather than a production spam filter.

VIII. CONCLUSION

This project implements and compares a hand-rolled Naive Bayes classifier against scikit-learn’s Bernoulli Naive Bayes, Logistic Regression, and SVM on the classic UCI Spambase dataset (4,601 rows, 57 features), alongside a separate raw-text Multinomial Naive Bayes pipeline evaluated on a 1,035-email corpus. On Spambase, Logistic Regression achieves the

strongest overall performance (0.9294 accuracy, 0.9702 AUC-ROC, 0.9246 ± 0.0068 CV accuracy), with SVM close behind, while the hand-rolled Naive Bayes implementation demonstrates the highest recall (0.9890) at the cost of lower precision (0.6950). These results reproduce, in a fully documented and reproducible pipeline, the well-known precision/recall trade-off between probabilistic and discriminative classifiers, while making explicit the log-space arithmetic and Laplace smoothing that underlie the Naive Bayes algorithm. The project is explicitly scoped as a pedagogical demonstration of the classical machine learning pipeline rather than a production-grade spam filter.